

Lecture 9

Introduction to Reinforcement Learning

AI for Geometry — MM845

Edward Hirst

Instituto de Matemática, Estatística e Computação Científica
Universidade Estadual de Campinas

`ehirst@unicamp.br`

Setembro 2026

Overview

1. Learning by Acting
2. Markov Decision Processes
3. Algorithms
4. RL for Mathematical Search
5. Summary

Recap (Lecture 8): Every embedding preserves some structure and destroys the rest
— PCA's honesty, t-SNE/UMAP's artefacts.

Guiding question: *When there is no dataset, but a score to assess local states, how can a machine learn to search?*

A Third Paradigm: No Dataset At All

Reinforcement Learning setting

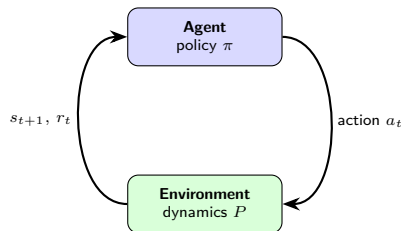
An **agent** repeatedly:

- (i) Observes a state s_t .
- (ii) Chooses an action a_t .
- (iii) Receives a reward r_t .
- (iv) Moves to s_{t+1} .

Goal: a *policy* (strategy) for choosing the next action, which maximises cumulative reward.

→ The data is generated by the agent's own behaviour, as evaluation on the states.

- Supervised: told the right answer.
Unsupervised: no answers.
RL: told only *how well things went* — evaluative, delayed, partial feedback.
- RL the right choice when:
 - (i) Decisions are *sequential*.
 - (ii) Feedback is a score.
 - (iii) Good examples are unavailable...i.e. **search and construction problems**.



The agent–environment loop: the only “dataset” is the trajectory the agent creates.

Geometric search:

state = a triangulation;

actions = flips;

reward = improvement of your favourite functional (e.g. FRST).

...RL learns *which* triangles to flip.

Can I iteratively perturb a starting solution towards an optimum solution?

The Formal Object

Definition (MDP)

A tuple (S, A, P, r, γ) :

state space S ; action space A ; transition kernel $P(s' | s, a)$; reward $r(s, a)$; discount $\gamma \in [0, 1)$.

A **policy** $\pi(a | s)$ selects actions; the objective is maximising expected discounted return

$$J(\pi) = \mathbb{E}_{\pi} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \right].$$

- **Markov property**: the future depends on the past only through the current state
- **Markov chain** has a fixed transition probability distribution: $s_{t+1} \sim P(\cdot | s_t)$.
- **MDP** is a controlled Markov chain, introducing a policy into the transition prob dist:
 $s_{t+1} \sim P^{\pi}(\cdot | s_t) = \sum_a \pi(a | s_t) P(\cdot | s_t, a)$. Each π a different Markov chain.
- An MDP may be thought of as a discrete-time dynamical system with input and cost.
- γ trades off present vs future; horizon scale $\sim (1 - \gamma)^{-1}$; guarantees series convergence.
- Geometric state/action spaces:
 - (i) *discrete* — triangulations with flips, knot diagrams with Reidemeister moves, graphs with edge edits.
 - (ii) *continuous* — points on a manifold with tangent directions, embeddings with deformations.
- Deterministic dynamics (most mathematical search problems) are special case $P \in \{0, 1\}$.

Value Functions and the Bellman Operator

Values

RL relies on computing (and approximating) the Value function, $V^\pi(s)$, and Action-Value function, $Q^\pi(s, a)$ (also "Q-function"):

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_t \gamma^t r_t \mid s_0 = s \right], \quad Q^\pi(s, a) = \mathbb{E}_\pi \left[\sum_t \gamma^t r_t \mid s_0 = s, a_0 = a \right].$$

Optimal values: $V^* = \sup_\pi V^\pi$, $Q^* = \sup_\pi Q^\pi$; an optimal policy acts greedily:

$$\pi^*(s) = \arg \max_a Q^*(s, a).$$

Bellman optimality equation — a fixed-point problem

$$(\mathcal{T}Q)(s, a) = r(s, a) + \gamma \mathbb{E}_{s' \sim P(\cdot | s, a)} \max_{a'} Q(s', a'), \quad Q^* = \mathcal{T}Q^*.$$

$\mathcal{T}$ is a γ -contraction in the sup norm $\Rightarrow$ (Banach) Q^* exists, is unique.

For any random starting Q , as a guess for Q^* , apply value iteration $Q_{k+1} = \mathcal{T}Q_k$, and this process will converge geometrically to Q^* .

- Dynamic programming (value/policy iteration) solves *known, small* MDPs exactly. RL proper begins when P is unknown (only sampled) or S is too large to enumerate — both true in interesting geometric search.

Three Ways to Solve an MDP

One question: which object do you represent?

The MDP hands you (S, A, P, r, γ) and asks for a policy. The families differ only in *what they store explicitly*; guarantees, admissible actions and sample cost all follow from that choice.

	Value-based	Policy-based	Model-based
learns	$Q_\theta(s, a)$	$\pi_\theta(a s)$	$\hat{P}, \hat{r}$
acts by	$\arg \max_a Q_\theta$	sampling π_θ	planning ahead
policy is	implicit	explicit	from search
needs	small discrete A	—	a model
e.g.	Q-learning, DQN	REINFORCE, PPO	AlphaZero, MuZero

- **Value-based:** *off-policy*, so every transition ever collected stays reusable (replay); but $\arg \max_a$ must be tractable, and its bootstrapping is what the deadly triad attacks.
- **Policy-based:** differentiate $J(\pi_\theta)$ directly, never forming values. Continuous and structured actions are native; the price is *on-policy* data (discarded after each update) and high variance.
- **Model-based:** cheapest in samples, and the only family that improves *at test time by thinking longer* — but a learned $\hat{P}$ is a surface the planner exploits precisely where it is wrong.

They compose — and geometry is handed one for free

Actor-critic = policy + value; AlphaZero = model + value + policy.

In mathematical search, often the model is *given* (P and r known), so making the method model-based is easy.

Tabular Q-Learning

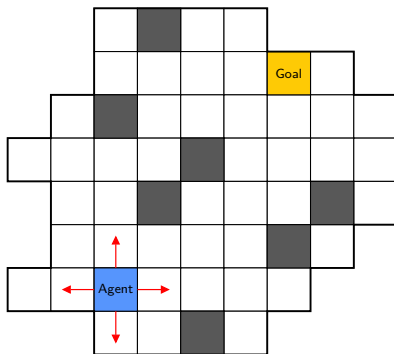
The algorithm

Maintain a table $Q(s, a)$. After experiencing (s, a, r, s') :

$$Q(s, a) \leftarrow \underbrace{\eta \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]}_{\text{bootstrap target}}$$

— a stochastic approximation of the Bellman fixed point, *from samples, without knowing P* .

- **Exploration:** behave ε -greedily (random action with prob. ε , else greedy); decay ε over training. No exploration $\Rightarrow$ never sees better options.
- Converges to Q^* if every pair (s, a) is visited infinitely often and steps decay suitably (Watkins–Dayan).
- *Off-policy:* learns the optimal policy while behaving exploratively.



Gridworld: states = cells, actions = moves, reward at the goal; obstacles in grey, – small enough to watch Q converge.

Scaling Up I: Deep Q-Networks

When the table is impossible

$|S|$ astronomically large (all triangulations, all graphs on n vertices, ...): replace the table by a network $Q_\theta(s, a)$ trained on the squared Bellman residual.

The deadly triad

function approximation + **bootstrapping** (targets contain Q_θ itself) + **off-policy data** — together they can diverge; RL loses the clean contraction guarantee the moment we leave the tabular world.

Standard stabilisers (DQN, 2015):

- **Replay buffer:** store transitions, train on random batches — decorrelates the stream, reuses data.
- **Target network:** compute bootstrap targets with a frozen copy θ^- , updated occasionally — stops the target chasing itself.
- Plus: reward clipping/normalisation, double-Q to correct the max-induced optimism bias.

Encode states with the right architecture

State = grid $\rightarrow$ CNN (L5); state = sequence $\rightarrow$ Transformer (L6); state = graph $\rightarrow$ GNN (L10).
The RL layer sits on top of everything already learned.

Scaling Up II: Policy Gradients & Continuous Actions

Optimise the policy directly

Parameterise $\pi_\theta(a \mid s)$

(e.g. a Gaussian over tangent directions — continuous actions handled natively).

Policy gradient theorem:

$$\nabla_\theta J(\pi_\theta) = \mathbb{E}_{\pi_\theta} \left[\sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t) G_t \right], \quad G_t = \text{return from time } t.$$

Reinforce actions in proportion to how well they turned out.

- **REINFORCE:** plug in sampled returns — unbiased but high variance.
- **Baselines:** subtract $V(s_t)$; the *advantage* $G_t - V(s_t)$ slashes variance without bias.
- **Actor-critic:** learn policy (actor) and value (critic) simultaneously; PPO also has clipped updates for stability — is the modern default and also the workhorse of LLM fine-tuning from feedback (Lecture 6 connection).
- **Continuous geometric control:** deforming an embedding/metric by choosing tangent vectors; step sizes as actions in optimisation on manifolds.

RL as a Conjecture-Refuting Machine

The template (Wagner 2021)

To refute " $\forall x \in \mathcal{X} : P(x)$ ": let the agent *build* x step by step (e.g. add an edge, choose a digit).

→ Reward = how close x comes to violating P

...measuring "how close" needs domain knowledge.

→ Train; inspect the best constructions

...several open conjectures in graph theory fell to exactly this loop.

- **Unknot recognition / knot simplification:** agent applies Reidemeister-type moves; reward for diagram simplification — learned unknotting strategies.
- **AlphaZero-style search:** policy/value networks guiding tree search — transferred from Go to matrix-multiplication algorithms (AlphaTensor) and program discovery; the pattern applies wherever moves compose.
- **Combinatorial geometry:** navigate flip graphs of triangulations, secondary fans, Pachner moves — optimise discrete curvature, count-type invariants, or find extremal examples.

Reward design is the mathematics

→ The agent optimises *exactly* what you wrote, not what you meant.

→ Smooth rewards train fast; sparse rewards (win/lose only) need enormous exploration.

→ Watch for **reward hacking**: degenerate solutions maximising the letter of the reward.

Practical Guidance — and When Not to Use RL

Tooling: gymnasium

- Allows definition of your state & action spaces explicitly and simply.
- Standardises environments (reset/step/reward API).
- Then any RL method / library plugs in and can run!

- **Escalation ladder** — try in order, measure, stop when solved:
 - ① Exhaustive / random search: sets a baseline from random actions.
 - ② Greedy / local search: use a prob dist over actions (e.g. simulated annealing).
 - ③ Evolutionary: mutate then select, many searches interacting in parallel.
...these are robust and gradient-free, (no MDP structure needed).
 - ④ RL: *sequential structure* matters and intermediate states inform decisions.
- **Known pain points:** sample-hungry (10^5 – 10^8 episodes), unstable across seeds (report several), sensitive to reward scale and hyperparameters, hard to reproduce — log everything (Lecture 1 hygiene).
- **Credit assignment:** delayed rewards make it hard to know which early move mattered, a deep reason RL is harder than supervised learning.

RL tuning is tough

- RL is a power tool for structured search.
- Expensive, temperamental, but occasionally capable of finding things nothing else finds!

Summary

Key points

- MDPs are problems defined by: (S, A, P, r, γ)
...the states, available actions, transition rule, reward, and future reward weighting.
- MDPs are controlled Markov chains; where a policy, π , chooses next action from current state, ideally which will maximise cumulative reward.
- Bellman's equation on MDPs is a contraction fixed-point problem
 $\implies$ tabular methods provably converge.
- RL introduces learning to MDPs, optimising from evaluative feedback on self-generated data; the paradigm for sequential search and construction.
- Three families, one distinction — *what you store*: **value-based** (Q_θ , act greedily), **policy-based** (π_θ , differentiate the return), **model-based** (dynamics + search).
- In mathematics the model is often *given and exact*: AlphaZero-style search is a natural template for RL as a counterexample/construction engine.
— but climb the escalation ladder first, and take time on reward design.

Next

Tutorial 9: Tabular Q-learning on discrete geometric structures.

Lecture 10: Geometry-aware ML — building symmetry into architectures.